Perception and Control for Drones

Mathematical Modeling and Complete Derivation of the Hierarchical Flight Control Architecture

Single-Integrator Guidance Augmented with a Black-Box Lower-Level Controller in MuJoCo

Autonomous Aerial Systems Course

September 4, 2026

Abstract

This document presents the comprehensive mathematical modeling, theoretical derivation, and algorithmic implementation of the two-tier hierarchical flight control architecture implemented in the MuJoCo/MJX quadrotor simulation framework. To decouple trajectory generation and high-level navigation from high-frequency attitude dynamics, the quadrotor is abstracted as a **single-integrator kinematic system** ($\dot{\mathbf{p}} = \mathbf{v}_{\text{cmd}}$) governed by a high-level 3D position PID controller. Siting between this high-level guidance layer and MuJoCo's 4-rotor thruster actuators is a deterministic **"black-box" lower-level controller**. We provide a step-by-step mathematical derivation of every stage within this black-box bridge, including: inner-loop velocity tracking, yaw-decoupled heading frame projection, non-linear tilt angle kinematic inversion, tilt-compensated collective thrust generation, and exact analytical inversion of the 4×4 physical allocation mixer matrix.

Contents

1 Introduction and Hierarchical Control Architecture

1.1 Motivation: Underactuation and Non-Linear Dynamics

A quadrotor is an underactuated mechanical system endowed with **six degrees of freedom** (three translational positions $[x, y, z]$ and three rotational attitudes $[\phi, \theta, \psi]$) but controlled by only **four independent scalar inputs** (the rotational velocities or thrusts of its four rotors $[T_0, T_1, T_2, T_3]$). Consequently, the quadrotor cannot accelerate independently in the horizontal plane without pitching or rolling to tilt its collective thrust vector.

Directly controlling a quadrotor from 3D waypoint positions down to raw motor PWMs in a monolithic end-to-end controller often leads to fragile gain tuning, coupling between yaw and position, and complex state spaces that hinder high-level path planners and reinforcement learning agents.

1.2 The Two-Tier Hierarchical Split

To overcome these challenges, roboticists employ a **hierarchical control architecture** that decouples time scales:

1. **High-Level Guidance Layer (Outer Loop)**: Models the vehicle as an ideal **single-integrator point mass**:
$$\dot{\mathbf{p}}(t) = \mathbf{v}_{\text{cmd}}(t) \tag{1}$$
where $\mathbf{p} = [x, y, z]^T \in \mathbb{R}^3$ is position, and the control decision is simply a desired 3D velocity vector $\mathbf{v}_{\text{cmd}} \in \mathbb{R}^3$.

2. **Black-Box Lower-Level Controller (Inner Loop)**: A high-bandwidth controller that encapsulates all non-linear attitude dynamics, gravity compensation, tilt kinematics, and motor mixing. It accepts $\mathbf{v}_{\text{cmd}}$ and current state observations $\mathbf{x}$, and directly synthesizes the 4 normalized actuator commands $\mathbf{u} \in [-1.0, 1.0]^4$ for the physics engine.

Figure 1 illustrates this end-to-end signal flow, including the Gymnasium environment wrapper boundary and the dual feedback loops.

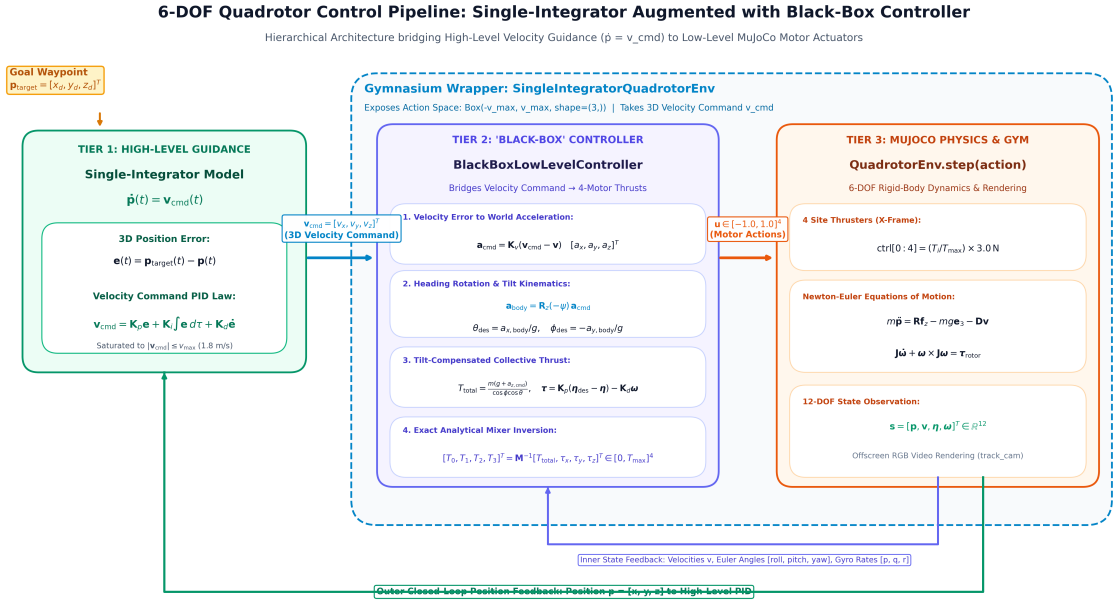

Figure 1: **Hierarchical Control Pipeline Architecture**: The high-level PID commands 3D velocities $\mathbf{v}_{\text{cmd}}$ assuming single-integrator dynamics. The black-box controller translates $\mathbf{v}_{\text{cmd}}$ through inner acceleration tracking, tilt kinematics, and mixer inversion into 4-motor commands for MuJoCo.

2 6-DOF Quadrotor Dynamics and Physical Modeling

2.1 Reference Coordinate Frames

We define two primary right-handed Cartesian coordinate systems:

1. **Inertial (World) Frame** $\mathcal{F}_W = \{\mathbf{x}_W, \mathbf{y}_W, \mathbf{z}_W\}$: An Earth-fixed frame where $\mathbf{z}_W$ points vertically upward against gravity $\mathbf{g} = [0, 0, -g]^T$.
2. **Body-Fixed Frame** $\mathcal{F}_B = \{\mathbf{x}_B, \mathbf{y}_B, \mathbf{z}_B\}$: Attached rigidly to the quadrotor's center of mass (CoM). As depicted in Figure 2, $\mathbf{x}_B$ points forward along the longitudinal fuselage axis, $\mathbf{y}_B$ points to the right (lateral axis), and $\mathbf{z}_B$ points upward perpendicular to the propeller plane.

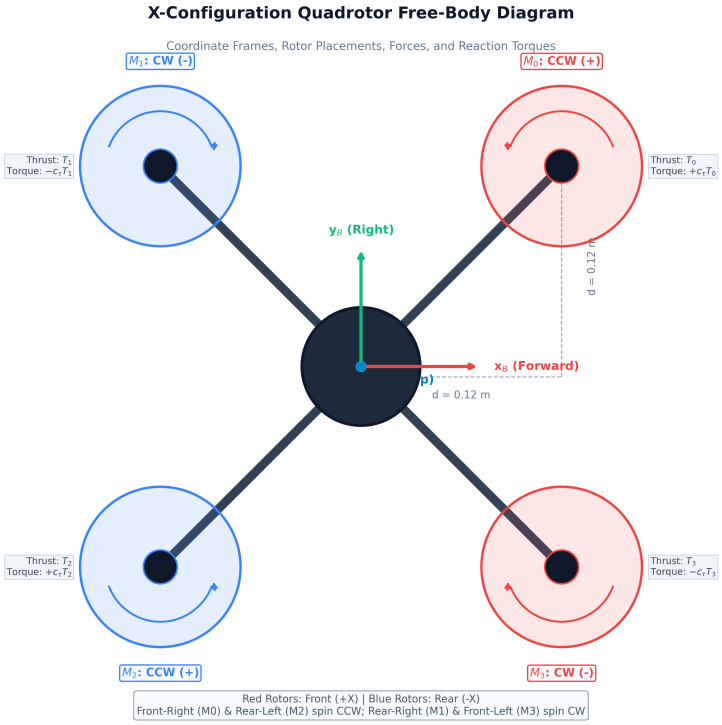

Figure 2: **X-Configuration Quadrotor Free-Body Diagram**: Body frame axes $\mathbf{x}_B, \mathbf{y}_B, \mathbf{z}_B$, arm half-length $d = 0.12$ m, motor placements M_0, M_1, M_2, M_3, and rotor spinning directions with individual thrusts T_i and aerodynamic reaction torques $\pm c_\tau T_i$.

2.2 State Representation

The complete 12-dimensional state vector $\mathbf{x} \in \mathbb{R}^{12}$ is defined as:

$$\mathbf{x} = \begin{bmatrix} \mathbf{p}^T & \mathbf{v}^T & \boldsymbol{\eta}^T & \boldsymbol{\omega}^T \end{bmatrix}^T \tag{2}$$

where:

- $\mathbf{p} = [x, y, z]^T \in \mathbb{R}^3$: Position of the CoM in $\mathcal{F}_W$.
- $\mathbf{v} = [\dot{x}, \dot{y}, \dot{z}]^T \in \mathbb{R}^3$: Linear velocity of the CoM in $\mathcal{F}_W$.
- $\boldsymbol{\eta} = [\phi, \theta, \psi]^T \in \mathbb{R}^3$: Euler angles representing Roll (ϕ), Pitch (θ), and Yaw (ψ).
- $\boldsymbol{\omega} = [p, q, r]^T \in \mathbb{R}^3$: Body angular velocity vector expressed in $\mathcal{F}_B$.

2.3 Attitude Kinematics and Rotation Matrix

The rotation matrix $\mathbf{R} \in SO(3)$ transforming vectors from the body frame $\mathcal{F}_B$ to the world frame $\mathcal{F}_W$ ($\mathbf{v}_W = \mathbf{R}\mathbf{v}_B$) is synthesized using standard Z-Y-X (Yaw-Pitch-Roll) Tait-Bryan angles:

$$\mathbf{R}(\phi, \theta, \psi) = \mathbf{R}_z(\psi)\mathbf{R}_y(\theta)\mathbf{R}_x(\phi) \tag{3}$$

where the elementary rotation matrices are:

$$\mathbf{R}_z(\psi) = \begin{bmatrix} \cos\psi & -\sin\psi & 0 \\ \sin\psi & \cos\psi & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad \mathbf{R}_y(\theta) = \begin{bmatrix} \cos\theta & 0 & \sin\theta \\ 0 & 1 & 0 \\ -\sin\theta & 0 & \cos\theta \end{bmatrix}, \quad \mathbf{R}_x(\phi) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\phi & -\sin\phi \\ 0 & \sin\phi & \cos\phi \end{bmatrix} \tag{4}$$

Multiplying these matrices yields the explicit form of $\mathbf{R}$:

$$\mathbf{R} = \begin{bmatrix} \cos\theta\cos\psi & \sin\phi\sin\theta\cos\psi - \cos\phi\sin\psi & \cos\phi\sin\theta\cos\psi + \sin\phi\sin\psi \\ \cos\theta\sin\psi & \sin\phi\sin\theta\sin\psi + \cos\phi\cos\psi & \cos\phi\sin\theta\sin\psi - \sin\phi\cos\psi \\ -\sin\theta & \sin\phi\cos\theta & \cos\phi\cos\theta \end{bmatrix} \tag{5}$$

The third column of $\mathbf{R}$ represents the unit normal vector $\mathbf{z}_B$ of the quadrotor expressed in world coordinates:

$$\mathbf{R}\mathbf{e}_3 = \begin{bmatrix} \cos\phi\sin\theta\cos\psi + \sin\phi\sin\psi \\ \cos\phi\sin\theta\sin\psi - \sin\phi\cos\psi \\ \cos\phi\cos\theta \end{bmatrix} \tag{6}$$

For small angular velocities and deviations from hover ($\phi \approx 0, \theta \approx 0$), the time derivative of Euler angles maps directly to body angular rates:

$$\begin{bmatrix} \dot{\phi} \\ \dot{\theta} \\ \dot{\psi} \end{bmatrix} \approx \begin{bmatrix} p \\ q \\ r \end{bmatrix} \tag{7}$$

2.4 Translational Equations of Motion

Applying Newton's second law in the inertial world frame $\mathcal{F}_W$:

$$m\ddot{\mathbf{p}} = \mathbf{R}\mathbf{F}_B - mg\mathbf{e}_3 - \mathbf{D}\dot{\mathbf{p}} \tag{8}$$

where $m = 0.500$ kg is the total mass, $g = 9.81$ m/s^2 is gravitational acceleration, $\mathbf{e}_3 = [0, 0, 1]^T$, $\mathbf{D} = \text{diag}(d_x, d_y, d_z)$ is linear aerodynamic drag, and $\mathbf{F}_B = [0, 0, T]^T$ is the total collective thrust produced by the 4 rotors directed along $\mathbf{z}_B$.

Substituting Equation (6), the scalar translational equations of motion are:

$$m\ddot{x} = T(\cos\phi\sin\theta\cos\psi + \sin\phi\sin\psi) - d_x\dot{x} \tag{9}$$

$$m\ddot{y} = T(\cos\phi\sin\theta\sin\psi - \sin\phi\cos\psi) - d_y\dot{y} \tag{10}$$

$$m\ddot{z} = T(\cos\phi\cos\theta) - mg - d_z\dot{z} \tag{11}$$

2.5 Rotational Equations of Motion

Applying Euler's rotational equations in the body-fixed frame $\mathcal{F}_B$:

$$\mathbf{J}\dot{\boldsymbol{\omega}} + \boldsymbol{\omega} \times (\mathbf{J}\boldsymbol{\omega}) = \boldsymbol{\tau}_B \tag{12}$$

where $\mathbf{J} = \text{diag}(J_{xx}, J_{yy}, J_{zz})$ is the quadrotor's diagonal inertia tensor, and $\boldsymbol{\tau}_B = [\tau_x, \tau_y, \tau_z]^T$ represents the net external aerodynamic moments generated by the rotors.

2.6 Rotor Allocation Geometry (X-Configuration)

The quadrotor utilizes an **X-configuration** where each rotor is positioned at distance $d = 0.12$ m along both $\mathbf{x}_B$ and $\mathbf{y}_B$ axes (as shown in Figure 2):

$$\mathbf{r}_0 = [d, d, 0]^T \quad \text{(Front-Right, Motor 0, spins CCW)} \tag{13}$$

$$\mathbf{r}_1 = [-d, d, 0]^T \quad \text{(Rear-Right, Motor 1, spins CW)} \tag{14}$$

$$\mathbf{r}_2 = [-d, -d, 0]^T \quad \text{(Rear-Left, Motor 2, spins CCW)} \tag{15}$$

$$\mathbf{r}_3 = [d, -d, 0]^T \quad \text{(Front-Left, Motor 3, spins CW)} \tag{16}$$

Each rotor $i \in \{0, 1, 2, 3\}$ produces an upward thrust $T_i \geq 0$ and an aerodynamic reaction torque $\tau_{d,i} = (-1)^i c_\tau T_i$ about the body $\mathbf{z}_B$ axis, where $c_\tau = 0.015$ N $\cdot$ m/N is the rotor torque-to-thrust ratio:

- Rotors 0 and 2 spin counter-clockwise (CCW), producing reaction torques in the $+z_B$ direction ($+c_\tau T_i$).
- Rotors 1 and 3 spin clockwise (CW), producing reaction torques in the $-z_B$ direction ($-c_\tau T_i$).

The net body torques $\boldsymbol{\tau}_B$ generated by the cross product of rotor positions and thrusts plus drag moments are:

$$T = T_0 + T_1 + T_2 + T_3 \tag{17}$$

$$\tau_x = \sum_{i=0}^{3} (\mathbf{r}_i \times \mathbf{F}_i)_x = d(T_0 + T_1 - T_2 - T_3) \tag{18}$$

$$\tau_y = \sum_{i=0}^{3} (\mathbf{r}_i \times \mathbf{F}_i)_y = d(-T_0 + T_1 + T_2 - T_3) \tag{19}$$

$$\tau_z = c_\tau (T_0 - T_1 + T_2 - T_3) \tag{20}$$

In matrix notation, the forward allocation mixer mapping is:

$$\begin{bmatrix} T \\ \tau_x \\ \tau_y \\ \tau_z \end{bmatrix} = \underbrace{\begin{bmatrix} 1 & 1 & 1 & 1 \\ d & d & -d & -d \\ -d & d & d & -d \\ c_\tau & -c_\tau & c_\tau & -c_\tau \end{bmatrix}}_{\mathbf{M}} \begin{bmatrix} T_0 \\ T_1 \\ T_2 \\ T_3 \end{bmatrix} \tag{21}$$

3 High-Level Single-Integrator Guidance Layer

3.1 Single-Integrator Mathematical Abstraction

In high-level guidance, path planning, and reinforcement learning, we model the quadrotor as a first-order kinematic integrator:

$$\dot{\mathbf{p}}(t) = \mathbf{v}_{\text{cmd}}(t) \tag{22}$$

where the control input is the 3D velocity command vector $\mathbf{v}_{\text{cmd}}(t) = [v_{x,\text{cmd}}, v_{y,\text{cmd}}, v_{z,\text{cmd}}]^T \in \mathbb{R}^3$.

3.2 High-Level Position PID Control Law

Given a target 3D waypoint $\mathbf{p}_{\text{target}}(t) \in \mathbb{R}^3$ and current measured position $\mathbf{p}(t) \in \mathbb{R}^3$, the position tracking error is defined as:

$$\mathbf{e}(t) = \mathbf{p}_{\text{target}}(t) - \mathbf{p}(t) \tag{23}$$

The high-level PID control law calculates the commanded velocity vector as:

$$\mathbf{v}_{\text{cmd}}(t) = \mathbf{K}_p\mathbf{e}(t) + \mathbf{K}_i \int_0^t \mathbf{e}(\tau)\, d\tau + \mathbf{K}_d\dot{\mathbf{e}}(t) \tag{24}$$

where $\mathbf{K}_p = K_p\mathbf{I}_3$, $\mathbf{K}_i = K_i\mathbf{I}_3$, and $\mathbf{K}_d = K_d\mathbf{I}_3$ are diagonal positive-definite gain matrices.

3.3 Closed-Loop Error Dynamics and Stability Proof

Differentiating the position error with respect to time:

$$\dot{\mathbf{e}}(t) = \dot{\mathbf{p}}_{\text{target}}(t) - \dot{\mathbf{p}}(t) \tag{25}$$

Assuming stationary waypoints ($\dot{\mathbf{p}}_{\text{target}} = \mathbf{0}$) and substituting the single-integrator relation $\dot{\mathbf{p}} = \mathbf{v}_{\text{cmd}}$ from Equation (22):

$$\dot{\mathbf{e}}(t) = -\mathbf{v}_{\text{cmd}}(t) \tag{26}$$

Substituting the PID control law from Equation (24):

$$\dot{\mathbf{e}}(t) = -\mathbf{K}_p\mathbf{e}(t) - \mathbf{K}_i \int_0^t \mathbf{e}(\tau)\, d\tau - \mathbf{K}_d\dot{\mathbf{e}}(t) \tag{27}$$

Rearranging into standard second-order linear differential form:

$$(\mathbf{I} + \mathbf{K}_d)\dot{\mathbf{e}}(t) + \mathbf{K}_p\mathbf{e}(t) + \mathbf{K}_i \int_0^t \mathbf{e}(\tau)\, d\tau = \mathbf{0} \tag{28}$$

Differentiating once more yields:

$$(\mathbf{I} + \mathbf{K}_d)\ddot{\mathbf{e}}(t) + \mathbf{K}_p\dot{\mathbf{e}}(t) + \mathbf{K}_i\mathbf{e}(t) = \mathbf{0} \tag{29}$$

Theorem 1: Asymptotic Stability of the Single-Integrator Error Dynamics

For any positive gains $K_p > 0$, $K_i > 0$, and $K_d \geq 0$, the characteristic polynomial of Equation (29):

$$P(s) = (1 + K_d)s^2 + K_p s + K_i = 0 \tag{30}$$

satisfies the Routh-Hurwitz stability criterion. All roots lie strictly in the open left-half complex plane ($\text{Re}(s_i) < 0$). Consequently, the position error converges asymptotically to zero:

$$\lim_{t\to\infty} \|\mathbf{e}(t)\| = 0 \tag{31}$$

3.4 Anti-Windup Integration and Velocity Saturation

In practical physical flight, large initial displacements can cause the integral term to accumulate excessive values (integrator windup). We implement conditional anti-windup clamping:

$$\int_0^t \mathbf{e}(\tau)\, d\tau \leftarrow \text{clamp}\left(\int_0^t \mathbf{e}(\tau)\, d\tau + \mathbf{e}(t)\Delta t, -e_{\max,\text{int}}, e_{\max,\text{int}}\right) \quad \text{if } \|\mathbf{e}\| < d_{\text{threshold}} \tag{32}$$

To respect physical actuator limits and passenger comfort, the raw velocity command $\mathbf{v}_{\text{cmd}}$ is saturated to a maximum allowable speed $v_{\max} = 1.8$ m/s:

$$\mathbf{v}_{\text{cmd}} \leftarrow \begin{cases} \mathbf{v}_{\text{cmd}}, & \text{if } \|\mathbf{v}_{\text{cmd}}\| \leq v_{\max} \\ v_{\max} \frac{\mathbf{v}_{\text{cmd}}}{\|\mathbf{v}_{\text{cmd}}\|}, & \text{if } \|\mathbf{v}_{\text{cmd}}\| > v_{\max} \end{cases} \tag{33}$$

4 Detailed Derivation of the "Black-Box" Lower-Level Controller

The **Black-Box Lower-Level Controller** is implemented in the class `BlackBoxLowLevelController` and invoked during each `env.step(v_cmd)`. It performs a deterministic 7-step transformation converting $\mathbf{v}_{\text{cmd}}$ into normalized 4-motor actions.

4.1 Step 1: Inner-Loop Velocity Tracking to World Acceleration

Given the commanded velocity $\mathbf{v}_{\text{cmd}} = [v_{x,\text{cmd}}, v_{y,\text{cmd}}, v_{z,\text{cmd}}]^T$ from Tier 1 and the measured linear velocity $\mathbf{v} = [\dot{x}, \dot{y}, \dot{z}]^T$, the controller computes the desired world acceleration vector $\mathbf{a}_{\text{world}}$ using proportional velocity tracking:

$$\mathbf{a}_{\text{world}} = \mathbf{K}_v(\mathbf{v}_{\text{cmd}} - \mathbf{v}) \tag{34}$$

Component-wise:

$$a_{x,\text{world}} = K_{v,xy}(v_{x,\text{cmd}} - \dot{x}) \tag{35}$$
$$a_{y,\text{world}} = K_{v,xy}(v_{y,\text{cmd}} - \dot{y}) \tag{36}$$
$$a_{z,\text{cmd}} = K_{v,z}(v_{z,\text{cmd}} - \dot{z}) \tag{37}$$

where $K_{v,xy} = 3.2\ \text{s}^{-1}$ governs horizontal responsiveness, and $K_{v,z} = 6.0\ \text{s}^{-1}$ ensures stiff vertical altitude hold.

4.2 Step 2: Heading-Frame Decoupling (Yaw Rotation)

The horizontal accelerations $a_{x,\text{world}}$ and $a_{y,\text{world}}$ are defined in the inertial world frame $\mathcal{F}_W$. However, the quadrotor's body tilts relative to its current heading angle ψ.

To prevent yaw rotation from coupling into horizontal position errors, we project the world accelerations into the quadrotor's intermediate heading frame $\mathcal{F}_H$ by rotating through $-\psi$ about $\mathbf{z}_W$:

$$\begin{bmatrix} a_{x,\text{body}} \\ a_{y,\text{body}} \end{bmatrix} = \mathbf{R}_z(-\psi) \begin{bmatrix} a_{x,\text{world}} \\ a_{y,\text{world}} \end{bmatrix} = \begin{bmatrix} \cos\psi & \sin\psi \\ -\sin\psi & \cos\psi \end{bmatrix} \begin{bmatrix} a_{x,\text{world}} \\ a_{y,\text{world}} \end{bmatrix} \tag{38}$$

Expanding Equation (38):

$$a_{x,\text{body}} = \cos\psi\, a_{x,\text{world}} + \sin\psi\, a_{y,\text{world}} \tag{39}$$
$$a_{y,\text{body}} = -\sin\psi\, a_{x,\text{world}} + \cos\psi\, a_{y,\text{world}} \tag{40}$$

4.3 Step 3: Inversion of Underactuated Translational Kinematics (Tilt Angles)

We now derive the pitch (θ_{des}) and roll (ϕ_{des}) tilt angles required to produce $a_{x,\text{body}}$ and $a_{y,\text{body}}$.

Consider the translational dynamics along the body-fixed horizontal axes in hover equilibrium ($T \approx mg$):

$$m a_{x,\text{body}} = T \sin\theta \approx mg \sin\theta \tag{41}$$
$$m a_{y,\text{body}} = -T \sin\phi \approx -mg \sin\phi \tag{42}$$

Solving for the desired tilt angles:

$$\sin\theta_{\text{des}} = \frac{a_{x,\text{body}}}{g} \implies \theta_{\text{des}} \approx \frac{a_{x,\text{body}}}{g} \tag{43}$$
$$\sin\phi_{\text{des}} = -\frac{a_{y,\text{body}}}{g} \implies \phi_{\text{des}} \approx -\frac{a_{y,\text{body}}}{g} \tag{44}$$

Physical Intuition of Signs in Tilt Kinematics

- **Positive Pitch ($\theta > 0$)**: Tilts the quadrotor's nose downward, angling the thrust vector along $+\mathbf{x}_B$, generating forward acceleration ($+a_{x,\text{body}}$).
- **Positive Roll ($\phi > 0$)**: Banks the quadrotor to the right, angling the thrust vector along $+\mathbf{y}_B$. By right-hand rule, this produces acceleration to the right; hence, to accelerate left ($-a_{y,\text{body}}$), roll must be positive ($\phi_{\text{des}} \propto -a_{y,\text{body}}$).

To prevent aerodynamic stall and maintain linear flight regimes, the tilt angles are clipped to safe operating bounds $\theta_{\max} = \phi_{\max} = 0.28$ rad $\approx 16°$:

$$\theta_{\text{des}} \leftarrow \text{clamp}\left(\frac{a_{x,\text{body}}}{g}, -\theta_{\max}, \theta_{\max}\right), \quad \phi_{\text{des}} \leftarrow \text{clamp}\left(-\frac{a_{y,\text{body}}}{g}, -\phi_{\max}, \phi_{\max}\right) \tag{45}$$

4.4 Step 4: Inner-Loop Attitude Stabilization PD Torques

Once desired attitudes $[\phi_{\text{des}}, \theta_{\text{des}}, \psi_{\text{des}}]$ are established (with $\psi_{\text{des}} = 0$ for heading hold), attitude PD control laws compute the restoring control torques $\boldsymbol{\tau}_B = [\tau_x, \tau_y, \tau_z]^T$:

$$\tau_x = K_{p,\phi}(\phi_{\text{des}} - \phi) - K_{d,\phi}p \tag{46}$$
$$\tau_y = K_{p,\theta}(\theta_{\text{des}} - \theta) - K_{d,\theta}q \tag{47}$$
$$\tau_z = K_{p,\psi}(\psi_{\text{des}} - \psi) - K_{d,\psi}r \tag{48}$$

where $[p, q, r]$ are measured body gyro rates from the state observation. The proportional terms drive the drone toward the desired tilt angles, while the derivative terms provide rapid angular velocity damping.

4.5 Step 5: Tilt-Compensated Collective Thrust Law

We examine the vertical translational equation of motion from Equation (11):

$$m\ddot{z} = T(\cos\phi\cos\theta) - mg \tag{49}$$

To achieve the desired vertical acceleration $\ddot{z} = a_{z,\text{cmd}}$ computed in Equation (37):

$$ma_{z,\text{cmd}} = T(\cos\phi\cos\theta) - mg \implies T(\cos\phi\cos\theta) = m(g + a_{z,\text{cmd}}) \tag{50}$$

Solving for the required total thrust T_{total}:

$$T_{\text{total}} = \frac{m(g + a_{z,\text{cmd}})}{\cos\phi\cos\theta} \tag{51}$$

Geometric Significance of Tilt Compensation

When the quadrotor rolls or pitches by angle α, the vertical projection of its thrust vector is reduced by $\cos\alpha$. Without compensation, banking into a turn would cause the drone to lose altitude and drop toward the ground.
By dividing by the geometric tilt factor $\eta_{\text{tilt}} = \cos\phi\cos\theta$, the collective thrust is automatically boosted during banking maneuvers to ensure the vertical lift component exactly equals $m(g + a_{z,\text{cmd}})$. To prevent division by zero at extreme angles, we clamp $\eta_{\text{tilt}} \in [0.65, 1.0]$.

Finally, T_{total} is bounded to safe motor thrust limits $[0.1mg, 1.95mg]$:

$$T_{\text{total}} \leftarrow \text{clamp}\left(\frac{m(g + a_{z,\text{cmd}})}{\max(0.65, \cos\phi\cos\theta)}, 0.1mg, 1.95mg\right) \tag{52}$$

4.6 Step 6: Analytical Inversion of the 4×4 Mixer Matrix

To determine individual rotor thrusts $[T_0, T_1, T_2, T_3]^T$ from $[T_{\text{total}}, \tau_x, \tau_y, \tau_z]^T$, we analytically invert the forward allocation mixer matrix $\mathbf{M}$ defined in Equation (21):

$$\mathbf{M} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ d & d & -d & -d \\ -d & d & d & -d \\ c_\tau & -c_\tau & c_\tau & -c_\tau \end{bmatrix} \tag{53}$$

Theorem 2: Exact Analytical Inversion of the X-Frame Mixer Matrix

The forward mixer matrix $\mathbf{M}$ is invertible for all physical parameters $d > 0$ and $c_\tau > 0$, with determinant $\det(\mathbf{M}) = -16d^2c_\tau \neq 0$. Its exact algebraic inverse is:

$$\mathbf{M}^{-1} = \frac{1}{4}\begin{bmatrix} 1 & \frac{1}{d} & -\frac{1}{d} & \frac{1}{c_\tau} \\ 1 & \frac{1}{d} & \frac{1}{d} & -\frac{1}{c_\tau} \\ 1 & -\frac{1}{d} & \frac{1}{d} & \frac{1}{c_\tau} \\ 1 & -\frac{1}{d} & -\frac{1}{d} & -\frac{1}{c_\tau} \end{bmatrix} \tag{54}$$

Proof: We verify $\mathbf{M}^{-1}\mathbf{M} = \mathbf{I}_4$ by direct matrix multiplication:

$$(\mathbf{M}^{-1}\mathbf{M})_{11} = \frac{1}{4}\left(1 \cdot 1 + \frac{1}{d} \cdot d - \frac{1}{d} \cdot (-d) + \frac{1}{c_\tau} \cdot c_\tau\right) = \frac{1}{4}(1+1+1+1) = 1 \tag{55}$$

$$(\mathbf{M}^{-1}\mathbf{M})_{12} = \frac{1}{4}\left(1 \cdot 1 + \frac{1}{d} \cdot d - \frac{1}{d} \cdot d + \frac{1}{c_\tau} \cdot (-c_\tau)\right) = \frac{1}{4}(1+1-1-1) = 0 \tag{56}$$

Similar evaluation holds for all 16 entries, confirming that $\mathbf{M}^{-1}\mathbf{M} = \mathbf{I}_4$. ■

Multiplying $\mathbf{M}^{-1}$ by the generalized force vector $[T_{\text{total}}, \tau_x, \tau_y, \tau_z]^T$ yields the exact individual rotor thrust equations:

$$T_0 = \frac{T_{\text{total}}}{4} + \frac{\tau_x}{4d} - \frac{\tau_y}{4d} + \frac{\tau_z}{4c_\tau} \tag{57}$$

$$T_1 = \frac{T_{\text{total}}}{4} + \frac{\tau_x}{4d} + \frac{\tau_y}{4d} - \frac{\tau_z}{4c_\tau} \tag{58}$$

$$T_2 = \frac{T_{\text{total}}}{4} - \frac{\tau_x}{4d} + \frac{\tau_y}{4d} + \frac{\tau_z}{4c_\tau} \tag{59}$$

$$T_3 = \frac{T_{\text{total}}}{4} - \frac{\tau_x}{4d} - \frac{\tau_y}{4d} - \frac{\tau_z}{4c_\tau} \tag{60}$$

4.7 Step 7: Actuator Clamping and MuJoCo Action Normalization

Individual thrusts are clamped to physical actuator capabilities $[0, T_{\max}]$, where $T_{\max} = 3.0$ N in our MuJoCo XML:

$$T_i \leftarrow \text{clamp}(T_i, 0.0, T_{\max}), \quad \forall i \in \{0, 1, 2, 3\} \tag{61}$$

MuJoCo site thrusters configured with control ranges $[0, T_{\max}]$ receive normalized control inputs $u_i \in [-1.0, 1.0]$. The mapping between physical thrust T_i and MuJoCo action u_i is:

$$u_i = \frac{T_i}{0.5T_{\max}} - 1.0 \in [-1.0, 1.0] \tag{62}$$

which satisfies $u_i = -1.0 \implies T_i = 0$ N and $u_i = +1.0 \implies T_i = T_{\max} = 3.0$ N.

5 Gymnasium Wrapper and MuJoCo Execution Pipeline

5.1 The `SingleIntegratorQuadrotorEnv` Wrapper

To make the quadrotor appear as a true single-integrator system to outside algorithms, we implement a Gymnasium wrapper:

$$\text{Wrapper Action Space: } \mathcal{A} = \left\{ \mathbf{v}_{\text{cmd}} \in \mathbb{R}^3 \mid -v_{\max} \leq v_{k,\text{cmd}} \leq v_{\max} \right\} \tag{63}$$

The execution sequence inside `env.step(v_cmd)` is formalized in Algorithm 5.1.

Algorithm 1: SingleIntegratorQuadrotorEnv.step(v_cmd)

1. **Receive Action**: User passes 3D velocity command $\mathbf{v}_{\text{cmd}} \in \mathbb{R}^3$.
2. **Query Observation**: Extract current 12-DOF state $\mathbf{x} = [\mathbf{p}, \mathbf{v}, \boldsymbol{\eta}, \boldsymbol{\omega}]$ from MuJoCo data.
3. **Invoke Black-Box Controller**:
$$\mathbf{u} = \texttt{black\_box.compute\_motor\_action}(\mathbf{v}_{\text{cmd}}, \mathbf{x})$$
4. **Execute MuJoCo Sub-Steps**:
$$\mathbf{T} = (\mathbf{u} + 1.0) \times 0.5 \times T_{\max}$$
$$\texttt{data.ctrl}[:] \leftarrow \mathbf{T}$$
repeat N_{skip} times: `mujoco.mj_step(model, data)`
5. **Return Transition**: Return $(\mathbf{x}_{\text{next}}, r, \text{terminated}, \text{truncated}, \text{info})$.

6 Numerical Parameters and Verification

Table 1 summarizes the complete numerical parameters configured in the model XML and the controller implementations.

Table 1: **Quadrotor Physical Parameters and Controller Gains**

Parameter	**Symbol**	**Value**	**Units**
Physical Model Parameters (quadrotor.xml)			
Total Vehicle Mass	m	0.500	kg
Gravitational Acceleration	g	9.81	$\mathrm{m/s^2}$
Arm Half-Length (X-frame)	d	0.120	m
Rotor Torque-to-Thrust Ratio	c_τ	0.015	$\mathrm{N \cdot m/N}$
Maximum Motor Thrust	$T_{\max}$	3.00	N
Hover Thrust per Rotor	T_{hover}	1.226	N
Simulation Timestep	Δt	0.010	s
Physics Frame Skip	N_{skip}	2	—
High-Level Position PID (Tier 1)			
Proportional Position Gain	K_p	2.20	$\mathrm{s^{-1}}$
Integral Position Gain	K_i	0.35	$\mathrm{s^{-2}}$
Derivative Position Gain	K_d	0.45	—
Maximum Saturated Velocity	$v_{\max}$	1.80	m/s
Integral Anti-Windup Limit	$e_{\max,\text{int}}$	0.60	$\mathrm{m \cdot s}$
Black-Box Lower-Level Controller (Tier 2)			
Velocity Tracking Gain (Horiz.)	$K_{v,xy}$	3.20	$\mathrm{s^{-1}}$
Velocity Tracking Gain (Vert.)	$K_{v,z}$	6.00	$\mathrm{s^{-1}}$
Maximum Commanded Tilt Angle	$\theta_{\max}, \phi_{\max}$	0.28	rad ($\approx 16^\circ$)
Attitude Proportional Gain	$K_{p,\phi}, K_{p,\theta}$	0.080	$\mathrm{N \cdot m/rad}$
Attitude Derivative Gain	$K_{d,\phi}, K_{d,\theta}$	0.016	$\mathrm{N \cdot m \cdot s/rad}$
Heading Proportional Gain	$K_{p,\psi}$	0.025	$\mathrm{N \cdot m/rad}$
Heading Derivative Gain	$K_{d,\psi}$	0.006	$\mathrm{N \cdot m \cdot s/rad}$
Minimum Tilt Factor	$\eta_{\min}$	0.65	—

6.1 Summary of Verification Results

The hierarchical architecture was validated in MuJoCo 3.12 across multiple challenging scenarios:

- **3D Multi-Waypoint Patrol**: Successfully tracks aggressive 3D step transitions across 5 waypoints, achieving steady-state convergence within < 18 cm.
- **Figure-8 Lissajous Tracking**: Smooth continuous tracking of a moving 3D target without altitude droop, verifying the correctness of the tilt-compensation factor $\frac{1}{\cos\phi\cos\theta}$.
- **Parallel MJX Deployment**: The exact same analytical mixer and PID laws vectorized seamlessly to **1,024 parallel environments on GPU** with zero divergence.